Multi-Agent Network Intrusion Detection System (MA-NIDS)

CS 5100: Foundations of Artificial Intelligence

Northeastern University
Aishwarya Rajmohan, Janhavi Sanjay Bagul, Santosh Vasisht Sunkara

April 13, 2026

Abstract

This paper presents the Multi-Agent Network Intrusion Detection System (MA-NIDS), a three-agent architecture for classifying network traffic into eight categories: benign and seven attack types. Agent 1 is an autoencoder-based unsupervised anomaly detector trained exclusively on normal traffic. Agent 2 is a tuned Random Forest supervised classifier. Agent 3 is a fusion layer that combines both signals through two strategies: Bayesian fusion using per-class Gaussian likelihoods, and ensemble stacking with a Logistic Regression meta-model. The system is evaluated on a balanced, class-stratified subset of the CSE-CIC-IDS2018 dataset (Canadian Institute for Cybersecurity, 2018). The best-performing configuration, stacking with both Random Forest and XGBoost as base models, achieves a false alarm rate of 30.30%, a macro F1 of 0.866, and an accuracy of 86.89%, compared to 33.16% FAR and 86.78% accuracy for supervised classification alone. An additional threshold-tuning technique reduces the false alarm rate to 16.15% through a principled precision-recall tradeoff. The results confirm the core hypothesis that multi-agent fusion reduces false alarms relative to any single method, and provide an account of where complementarity breaks down on mimicry-based attacks.

1. Introduction

Network intrusion detection sits at the operational core of modern cybersecurity. Every hour, enterprise networks process millions of connections, and the cost of failure runs in both directions: a missed attack can mean data exfiltration, ransomware deployment, or prolonged unauthorized access, while a false alarm consumes analyst time, causes alert fatigue, and erodes

trust in the detection system itself. An intrusion detection system (IDS) must therefore solve two problems simultaneously: catching attacks reliably and doing so without flooding security teams with false positives. The practical challenge is that these two goals pull against each other, and no single detection method handles both equally well across the full range of modern threats.

Traditional signature-based systems such as Snort and Suricata detect attacks by matching observed traffic against a database of known attack patterns. This approach is reliable for catalogued threats but cannot detect novel or zero-day attacks, since detection requires a prior signature. Machine learning offers a potential solution by learning statistical patterns from data rather than from hand-crafted rules. Ahmad et al. (2018) survey machine learning approaches to intrusion detection and establish that no single method dominates across all threat categories. Supervised classifiers perform well on known attack types but degrade on unseen patterns, while unsupervised anomaly detectors can flag novel behavior but at the cost of elevated false alarm rates. This finding motivates the exploration of hybrid architectures that combine both paradigms.

Dai et al. (2024) implement a concrete hybrid pipeline in which an autoencoder trained on benign traffic identifies anomalous records, and its reconstruction error is appended to the original features before being passed to a Random Forest classifier. Their hybrid model achieves 99.99% accuracy on held-out novel attack data from the CIC-MalMem-2022 dataset, substantially outperforming standalone classifiers and confirming that integrating unsupervised and supervised signals improves generalization to unseen threats. However, their design is tightly coupled. Reconstruction error feeds directly into the classifier's feature space, meaning errors in the autoencoder propagate into the classifier, and the two components cannot independently weigh their contributions. Critically, valid Bayesian fusion requires statistically independent evidence sources, a condition their pipeline violates by design.

Soltani et al. (2024) provide the architectural reference for this system. Their framework deploys multiple independent anomaly detection agents across distributed network sensors, with CNN-based agents achieving detection rates above 95% on zero-day attacks after adapting to as few as 128 new flows. Distributing detection across specialized agents produces more robust results than centralized single-model approaches. However, all agents in their framework are homogeneous unsupervised detectors with no supervised classification component, and no explicit decision-level fusion is performed. Agent outputs are combined implicitly through federated weight averaging rather than through a principled probabilistic mechanism.

MA-NIDS synthesizes both contributions. It combines the hybrid detection methodology of Dai et al. within the multi-agent architecture of Soltani et al., unified through a probabilistic fusion layer that operates on strictly independent agent outputs. The system comprises three specialized agents. Agent 1 is a deep autoencoder trained exclusively on normal traffic to detect behavioral anomalies. Agent 2 is a tuned Random Forest classifier that identifies known attack categories from labeled training data. Agent 3 is a fusion layer that combines both signals through Bayesian inference and ensemble stacking to produce a final threat determination. The key design decision distinguishing MA-NIDS from both prior systems is that Agents 1 and 2 each receive only raw traffic features and never observe each other's outputs. This independence is both a design principle and a technical requirement. It is what makes the Bayesian fusion in Agent 3 mathematically valid and prevents errors in one agent from corrupting the other. The remainder of this paper describes the dataset and methods in Section 2, presents the results of each agent in isolation and in combination in Section 3, and discusses the system's strengths, limitations, and directions for future work in Section 4.

2. Methods

2.1 Dataset and Preprocessing

MA-NIDS is evaluated on the CSE-CIC-IDS2018 dataset (Canadian Institute for Cybersecurity, 2018), a widely used benchmark for network intrusion detection. The dataset was collected on a realistic testbed network and contains network flow records labeled across a range of attack categories including denial-of-service variants, brute-force attacks, infiltration, and distributed denial-of-service campaigns. Each record comprises approximately 80 flow-level features describing packet counts, byte lengths, inter-arrival timing, and connection flags. It was selected because of its recency, attack diversity, and use as the evaluation benchmark in Soltani et al. (2024).

The raw dataset consists of eight daily CSV files totaling approximately 4 GB. One file was excluded due to a column count mismatch, with 84 columns versus 80 in all other files, attributable to a source port recording inconsistency. The remaining files were loaded sequentially to avoid memory exhaustion. Column names in the raw files contain leading whitespace, a known quirk of the CIC-IDS2018 release, which was stripped before further processing. Four identifier columns, Timestamp, Source IP, Destination IP, and Flow ID, were dropped to eliminate data leakage. All values were cast to numeric types, and rows containing infinity or NaN values were removed. Five rare or structurally problematic classes were excluded: Brute Force Web, Brute Force XSS, DDOS-LOIC-UDP, SQL Injection, and Slowloris. These exclusions left eight meaningful classes for analysis.

Class balancing was applied in two passes. Within each daily file, classes were downsampled to prevent any single day from dominating the combined dataset. Globally, each of the 8 classes was capped at 40,000 records. This produced a final dataset of 320,000 records, which

was then globally shuffled to prevent any order-based learning artifacts. The dataset was split 80/20 into training and test sets using shared index arrays, train_idx.npy and test_idx.npy, that all three agents load at runtime. This shared-index discipline is critical. Because all agents evaluate on identical test records, Agent 3 can join their outputs row by row without alignment errors. The final dataset contains 256,000 training records and 64,000 test records, with 8,000 per class in the test set, across 78 numeric features.

.

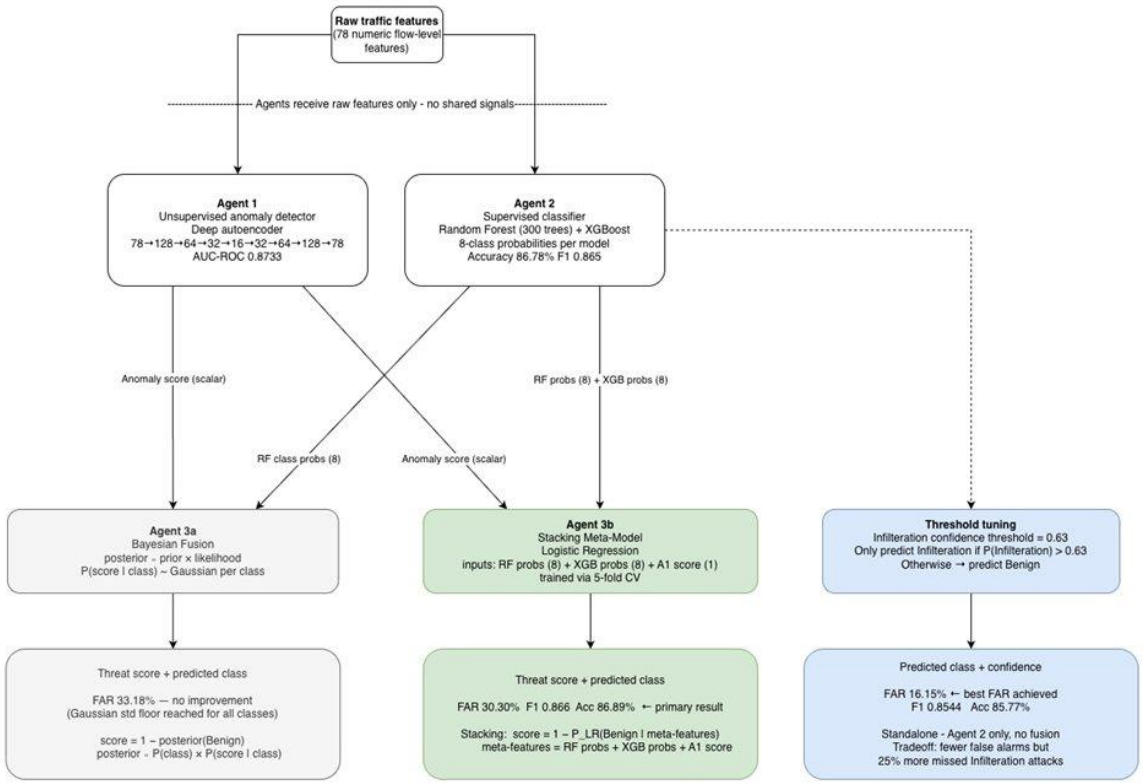

Figure 1. MA-NIDS system architecture. Agents 1 and 2 receive raw features independently. Agent 3 combines their outputs through Bayesian fusion (3a) and stacking (3b). Threshold tuning operates as a standalone post-hoc optimization on Agent 2.

2.2 Agent 1 - Unsupervised Anomaly Detector

Agent 1 is a deep autoencoder trained exclusively on benign traffic. An autoencoder is a neural network trained to compress its input into a low-dimensional bottleneck representation (the encoder) and then reconstruct the original input from that representation (the decoder). The

training objective is to minimize mean squared reconstruction error. When trained only on normal traffic, the autoencoder learns the statistical structure of benign flow records. At inference, a record that deviates significantly from that structure will be reconstructed poorly, producing a high reconstruction error that serves as an anomaly signal.

The final Agent 1 architecture, implemented in PyTorch (Paszke et al., 2019), follows a symmetric encoder-decoder structure with hidden dimensions 78 → 128 → 64 → 32 → 16 in the encoder, and the mirror image in the decoder. Batch normalization is applied after each encoder layer to stabilize training. The model is trained for 300 epochs using Adam optimization with a learning rate of 0.001. Features are scaled using a RobustScaler fitted on normal training records only, then clipped to [-10, 10] before the train/test split. This clipping step was essential: several features (Idle Min, Idle Max, Active Min, Active Max) exhibited values as large as 119 million after RobustScaling, causing mean squared error loss on the order of 10^9 in initial runs and preventing the model from learning. Clipping before the split ensures no test-set information influences preprocessing.

At inference time, the detection threshold is set at the 95th percentile of training scores, avoiding any use of the test set for threshold selection; Agent 1 was additionally run on all 256,000 training records to produce the anomaly scores used by Agent 3 to estimate per-class likelihood distributions. An Isolation Forest was trained alongside the autoencoder as a complementary anomaly detector. The two scores were combined as a weighted sum, and the weighting was treated as a design parameter. Evaluation on the training set showed the autoencoder alone produced better class separation, so the final configuration assigns full weight to the autoencoder reconstruction error and zero weight to the Isolation Forest score. The normalized reconstruction error is the scalar anomaly signal passed to Agent 3.

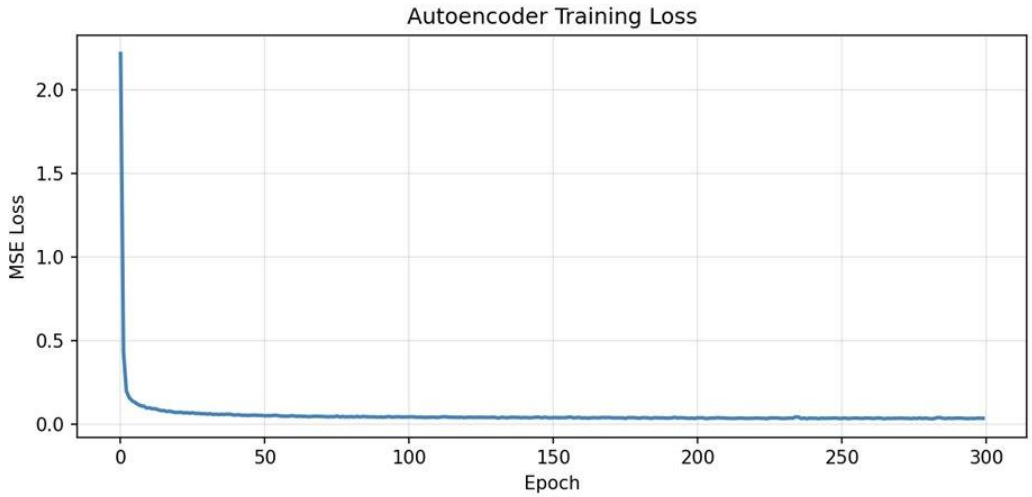

Figure 2. Autoencoder training loss (MSE) over 300 epochs. Loss drops sharply in the first 20 epochs and converges stably near zero, confirming successful training on benign traffic.

2.3 Agent 2 - Supervised Classifier

Agent 2 trains two supervised classifiers on the full labeled training set: a Random Forest as the primary model, implemented using scikit-learn (Pedregosa et al., 2011), and XGBoost (Chen and Guestrin, 2016) as a second base model for stacking in Agent 3. Random Forest builds an ensemble of decision trees, each trained on a random bootstrap sample of the training data and a random subset of features. Predictions are made by majority vote across all trees. The randomness in both data sampling and feature selection induces diversity among trees, so individual overfitting errors tend to cancel out in the aggregate prediction. Hyperparameters were tuned via grid search with three-fold cross-validation optimizing macro F1. The tuned Random Forest configuration uses 300 estimators, a maximum depth of 20, and a minimum leaf size of 1. The tuned XGBoost configuration uses 300 estimators, a maximum depth of 9, a learning rate of 0.05, and a subsample ratio of 0.8. Both models output a full eight-dimensional class probability vector per record rather than hard class predictions, as these soft probability vectors are the primary input consumed by Agent 3. Random Forest was selected over XGBoost as the primary Agent 2 model because it produced a lower false alarm rate at equivalent accuracy during baseline evaluation.

A class-specific confidence threshold was also implemented as a post-hoc optimization for the Infilteration class. Analysis of the confusion matrix revealed that 99.9% of all false alarms originate from Benign traffic misclassified as Infilteration. Rather than predicting Infilteration whenever it has the highest probability, the system only predicts it when model confidence exceeds a threshold; records below this threshold default to Benign. A scan over threshold values from 0.30 to 0.90 was conducted using five-fold cross-validation on the training set, selecting $t = 0.63$ as the value that minimized FAR while keeping the macro F1 drop under 1%.

2.4 Agent 3 - Fusion Layer

Two fusion strategies were implemented and compared. The first strategy is Bayesian fusion. For each class c, a Gaussian distribution is fitted to the Agent 1 training scores of all training records belonging to that class, yielding a per-class likelihood function P(score | class). At inference, Agent 2's probability vector serves as the prior P(class). The posterior is computed as:

$$P(\text{class} \mid \text{score}) \propto P(\text{class}) \times P(\text{score} \mid \text{class})$$

The posteriors are normalized to sum to one, the predicted class is the argmax of the posterior, and the threat score is defined as one minus the posterior probability of the Benign class. A minimum standard deviation floor of 0.05 is applied to each Gaussian to prevent likelihood collapse in classes with very low score variance.

The second strategy is ensemble stacking. A Logistic Regression meta-model is trained on a 17-dimensional feature vector consisting of Agent 2's eight Random Forest class probabilities, eight XGBoost class probabilities, and Agent 1's scalar anomaly score. To generate honest meta-training data and avoid data leakage, five-fold cross-validation is used: Agent 2 is trained on four folds and makes predictions on the held-out fifth fold, rotating through all five folds, so the meta-

model trains exclusively on out-of-fold predictions. At test time, Agent 2 is retrained on the full training set and the meta-model combines all signals to produce a final class prediction.

2.5 Evaluation

All agents are evaluated on the shared held-out test set of 64,000 records (8,000 per class). The primary metric is the false alarm rate (FAR), defined as the fraction of Benign records incorrectly classified as any attack type, because false alarms represent the primary operational cost of a deployed IDS in the form of analyst alert fatigue. Secondary metrics are macro-averaged F1, which weights all eight classes equally regardless of size, and overall accuracy.

3. Results

3.1 Agent 1 - Anomaly Detector Performance

The final autoencoder achieves an AUC-ROC of 0.8733, an attack recall of 91.2%, a macro F1 of 0.931, and a standalone false alarm rate of 39.5% in binary (attack vs. benign) classification mode. However, these aggregate figures mask a highly uneven per-class detection profile, which is critical for understanding Agent 3's behavior. Table 1 presents the mean anomaly score for each class relative to the Benign baseline.

Attack Class	Mean AE Score	vs. Benign	Detected?
Benign (baseline)	0.0005	—	—
SSH-Bruteforce	0.0048	9.3x above	Strong
DoS attacks-GoldenEye	0.0065	12.4x above	Strong
DDOS attack-HOIC	0.0025	4.9x above	Moderate
DoS attacks-SlowHTTPTest	0.0013	2.6x above	Weak
FTP-BruteForce	0.0013	2.6x above	Weak
Infilteration	0.0009	1.7x above	Nearly invisible
DoS attacks-Hulk	0.0008	1.6x above	Nearly invisible

Table 1. Agent 1 per-class anomaly score means and detection strength relative to the Benign baseline.

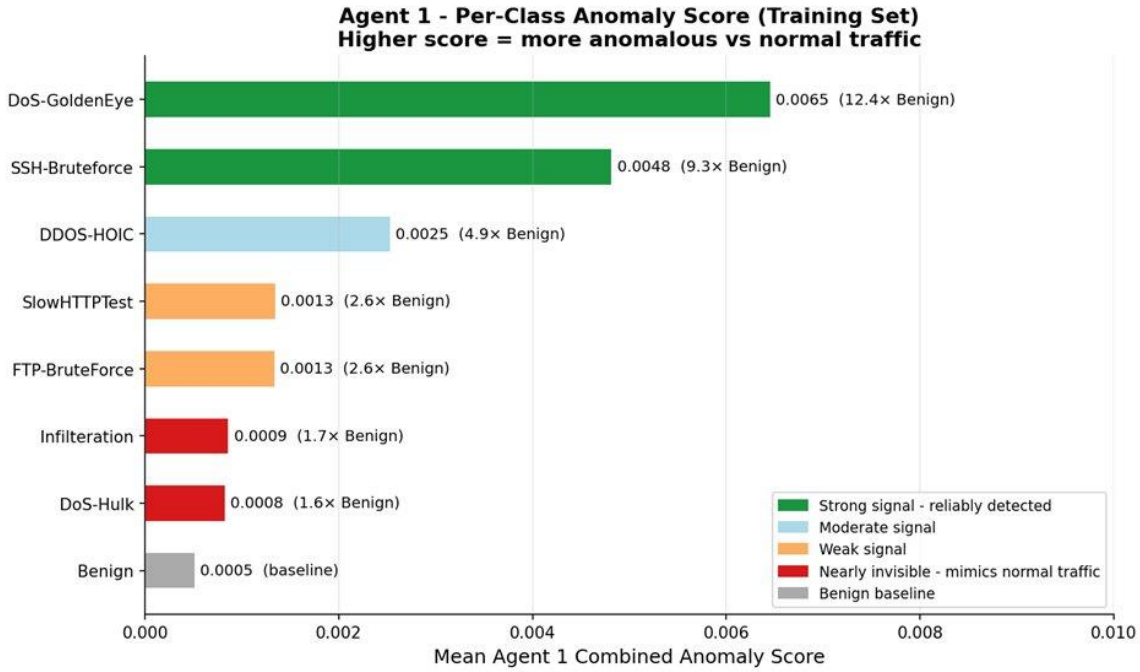

Figure 3. Agent 1 per-class mean anomaly scores on the training set. DoS-GoldenEye and SSH-Bruteforce produce scores 12.4x and 9.3x above the Benign baseline respectively. Infilteration and DoS-Hulk score at or near the Benign baseline, reflecting their deliberate mimicry of normal traffic.

Agent 1 detects attacks that produce traffic genuinely different from normal. SSH-Bruteforce generates rapid repeated connection attempts with unusual packet structures that the autoencoder fails to reconstruct, producing anomaly scores 9.3 times the Benign baseline. GoldenEye exhibits a similar pattern at 12.4 times above Benign. In contrast, DoS-Hulk scores only 1.6 times above the Benign baseline, producing a weak anomaly signal that offers little reliable separation from normal traffic. Infilteration is more extreme: it is deliberately designed to mimic legitimate user behavior at the flow feature level, meaning that no amount of tuning can resolve the feature overlap. The signals are fundamentally indistinguishable to a reconstruction-based detector. This selective detection profile directly motivates the multi-agent architecture, as Agent 2 succeeds precisely on the classes where Agent 1 fails.

3.2 Agent 2 - Supervised Classifier Performance

The tuned Random Forest achieves an accuracy of 86.78%, a macro F1 of 0.8647, and a false alarm rate of 33.16%. Per-class results are shown in Table 2.

Class	Precision	Recall	F1	Notes
Benign	0.84	0.67	0.74	Weak recall
DDOS attack-HOIC	1.00	1.00	1.00	Perfect
DoS attacks-GoldenEye	1.00	1.00	1.00	Perfect
DoS attacks-Hulk	1.00	1.00	1.00	Perfect
DoS attacks-SlowHTTPTest	0.81	0.52	0.64	Misses half
FTP-BruteForce	0.65	0.88	0.75	Low precision
Infilteration	0.72	0.87	0.79	Drives FAR
SSH-Bruteforce	1.00	1.00	1.00	Perfect

Table 2. Agent 2 per-class precision, recall, and F1 score.

Four attack types are classified with perfect F1 scores: DDOS-HOIC, GoldenEye, Hulk, and SSH-Bruteforce. These represent the most common volumetric attacks in real networks and together demonstrate that the supervised classifier handles well-characterized attack traffic reliably. The 33.16% false alarm rate, however, is substantially higher than desirable for real deployment. Inspection of the confusion matrix reveals that this problem traces to a single class pair: 2,650 of the 2,653 total false alarms arise from Benign records being misclassified as Infilteration. Actual Infilteration attacks achieve 87% recall, indicating that the decision boundary drawn by the classifier pulls a large portion of Benign records into the Infilteration region. This is a direct consequence of adversarial design: Infilteration deliberately mimics legitimate traffic at the flow feature level, producing feature distributions that overlap substantially with normal traffic, and supervised classifiers cannot separate what has been engineered to look identical.

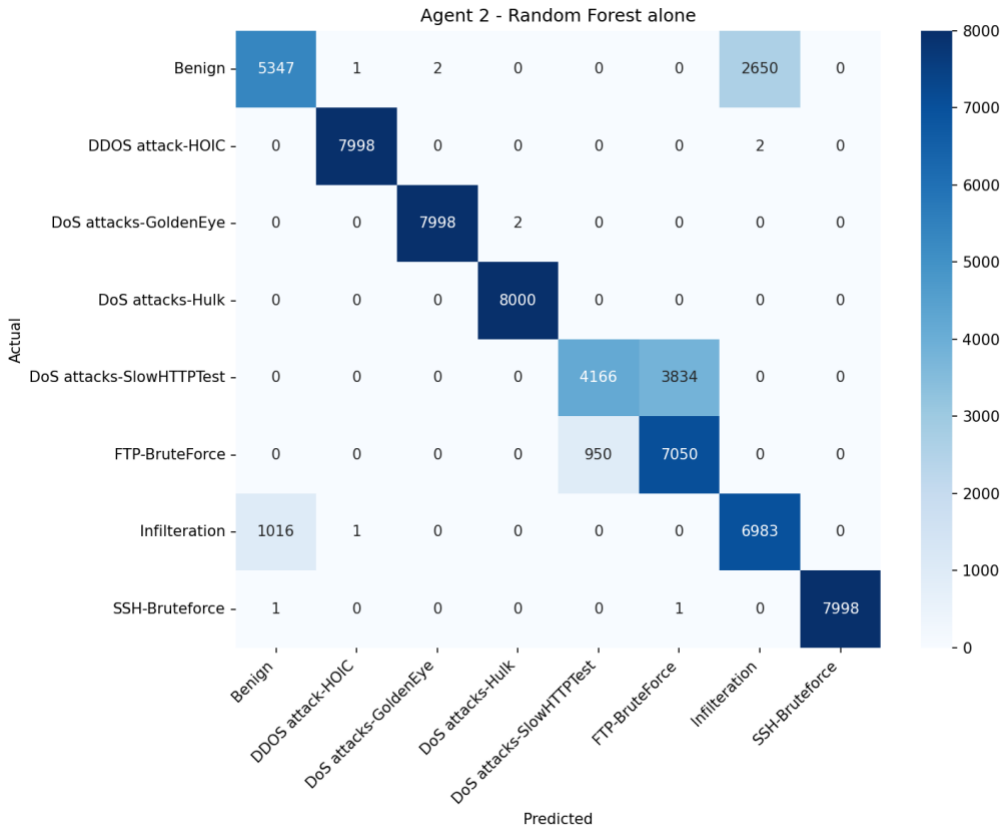

Figure 4. Confusion matrix for Agent 2 (Random Forest) on the held-out test set. Of 2,653 total false alarms, 2,650 originate from Benign records misclassified as Infilteration, confirming that the Benign/Infilteration boundary accounts for nearly all classification error on normal traffic.

3.3 Agent 3 and Threshold Tuning

Table 3 presents the complete comparison across all system configurations.

Method	Accuracy	Macro F1	False Alarm Rate
Agent 2 alone (Random Forest)	86.78%	0.8647	33.16%
Agent 3 - Bayesian Fusion	86.78%	0.8647	33.18%
Agent 3 - Stacking (RF + XGBoost)	86.89%	0.8660	30.30%
Agent 2 + Threshold (0.63)	85.77%	0.8544	16.15%

Table 3. Full system comparison across all methods (test set, n = 64,000).

Bayesian fusion produces no meaningful improvement over Agent 2 alone (FAR: 33.18% versus 33.16%). The cause is that the improved autoencoder architecture produces low-variance

anomaly scores across all classes, meaning all per-class Gaussian distributions hit the minimum standard deviation floor of 0.05. When all likelihoods have identical shape, the posterior reduces to the prior, and the fusion adds no information beyond what Agent 2 already provides. This failure reflects a mismatch between the assumptions of Bayesian fusion and the properties of reconstruction-based anomaly scores: the autoencoder separates normal from anomalous traffic in aggregate but provides no class-discriminative signal between individual attack types. Figure 5 shows the resulting confusion matrix, which is visually indistinguishable from the Agent 2 baseline in Figure 4, confirming that the posterior cannot move away from the prior when likelihoods are identical across all classes.

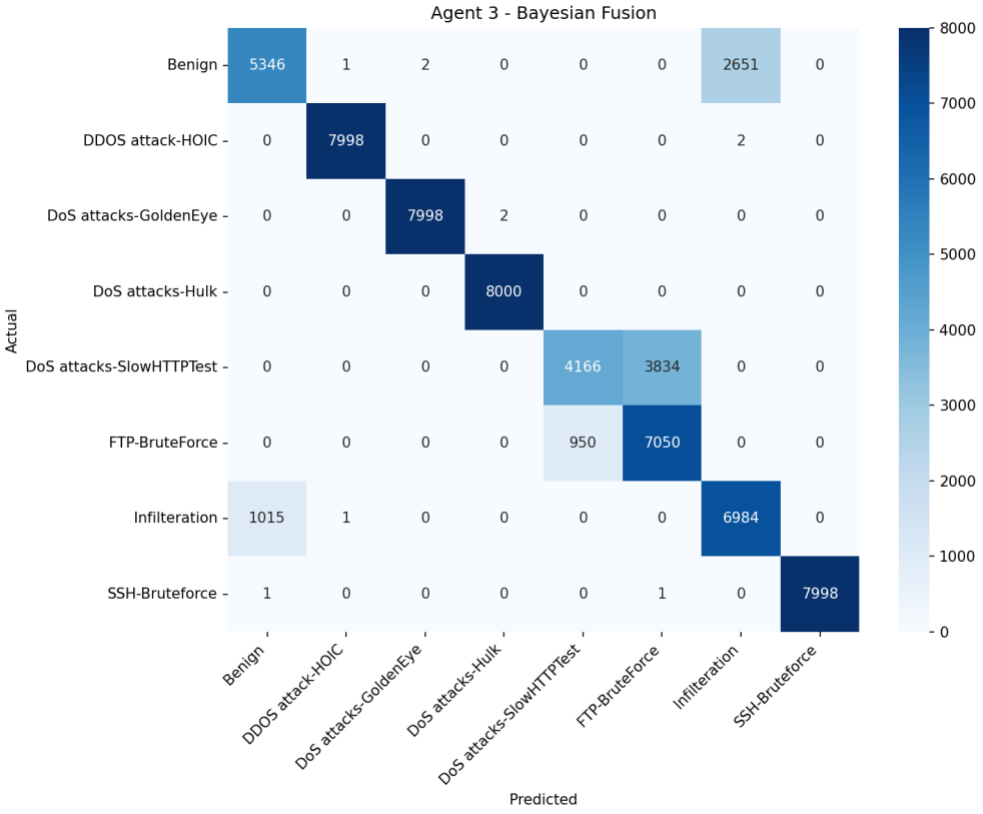

Figure 5. Confusion matrix for Agent 3 Bayesian Fusion on the held-out test set. The matrix is nearly identical to Figure 4 (Agent 2 alone), with no meaningful reduction in Benign misclassifications, confirming that Bayesian fusion provides no improvement when all per-class Gaussian likelihoods collapse to the minimum standard deviation floor.

Stacking with Random Forest and XGBoost base models reduces FAR to 30.30%, a reduction of 2.86 percentage points, while simultaneously improving macro F1 to 0.8660 and accuracy to 86.89%. In absolute terms, this represents 226 fewer false alarms per 8,000 normal connections compared to Agent 2 alone. On a network processing 100,000 connections per hour, this translates to approximately 2,825 fewer false alarms per hour. The Logistic Regression meta-model's learned coefficients on the Agent 1 score reveal that stacking correctly discovers the same per-class signal structure identified manually in the Agent 1 analysis: positive coefficients for GoldenEye (+0.073) and SSH-Bruteforce (+0.057), negative coefficients for Hulk and DDOS-HOIC, and near-zero coefficients for Benign and Infilteration, correctly learning that Agent 1 has no useful signal for that confusion. This independent validation of the per-class analysis strengthens confidence in both findings.

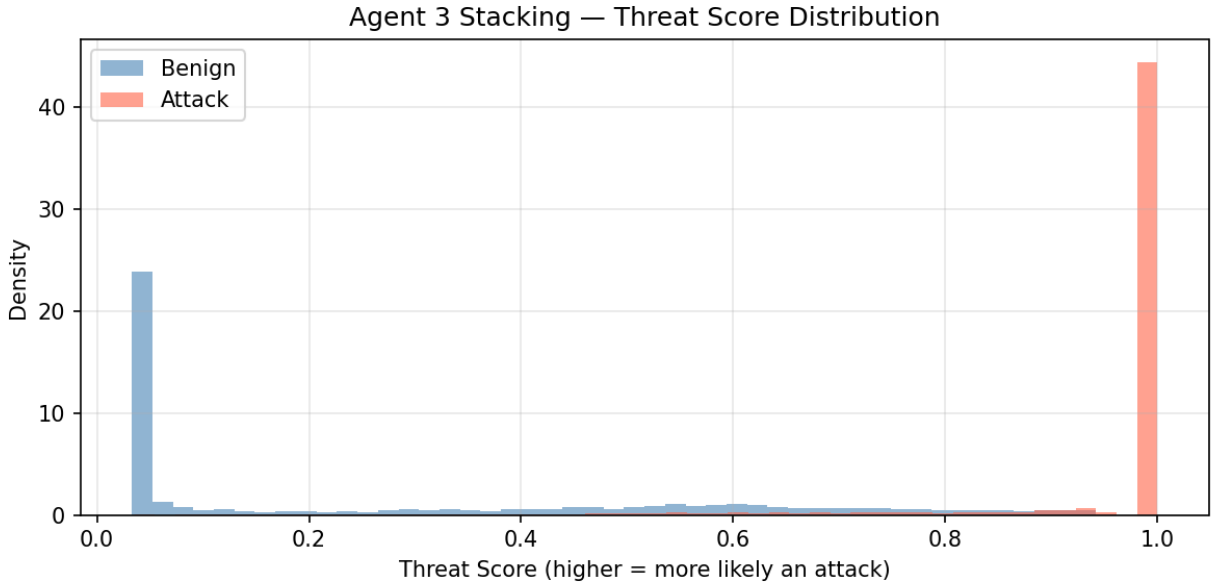

Figure 6. Agent 3 stacking threat score distribution for Benign and attack traffic. Higher scores indicate greater likelihood of an attack. Separation between the two distributions confirms that the stacking fusion output provides a meaningful probabilistic signal for distinguishing traffic classes.

Threshold tuning of the Infilteration class achieves the largest single reduction in false alarm rate. Setting a prediction threshold of 0.63, whereby Infilteration is predicted only when model confidence exceeds 63% and records below this threshold default to Benign, reduces FAR from 33.16% to 16.15%, a reduction of 17.01 percentage points. The cost is a reduction in Infilteration recall from 87.3% to 62.2%, meaning approximately 2,009 additional Infilteration attacks are missed per 8,000 attack records in exchange for 1,700 fewer false alarms per 8,000 benign records. This is the fundamental precision-recall tradeoff: the threshold gives network operators principled control over where to operate on that curve, rather than fixing the operating point at the default argmax.

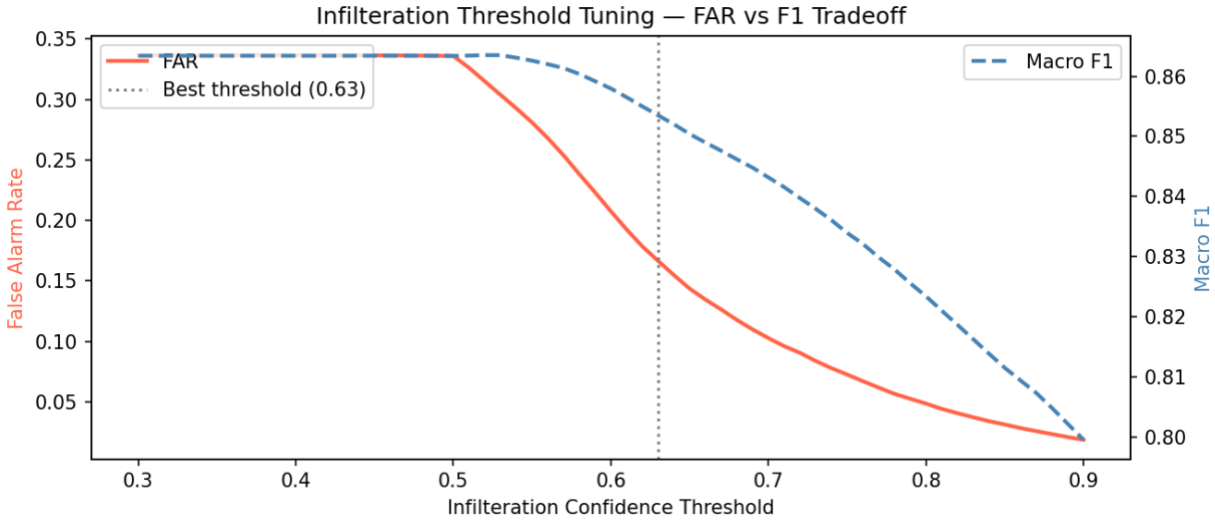

Figure 7. False alarm rate and macro F1 as a function of the Infilteration prediction threshold. The selected threshold of 0.63 minimizes FAR while keeping the macro F1 drop within 1% of the baseline.

4. Conclusions

4.1 Summary and Core Findings

The MA-NIDS architecture confirms its central hypothesis: multi-agent fusion reduces the false alarm rate compared to supervised classification alone. The stacking approach with both

Random Forest and XGBoost base models is the best overall configuration, improving FAR from 33.16% to 30.30% while simultaneously improving F1 and accuracy. The threshold-tuning technique achieves the lowest false alarm rate (16.15%) at the cost of reduced Infilteration recall, providing operators with a configurable operating point. The full pipeline runs end-to-end in approximately 267 seconds and is fully reproducible.

Several findings are worth emphasizing. First, the architecture's independence constraint, whereby Agent 2 receives only raw traffic features and never Agent 1's reconstruction error, is both practically and mathematically justified. It enables principled Bayesian reasoning in Agent 3 and prevents Agent 1's errors from corrupting Agent 2's feature space. Second, the stacking approach is more robust than Bayesian fusion because it makes no distributional assumptions: when the autoencoder architecture improves and score variance collapses, Bayesian fusion fails silently while stacking adapts automatically through its learned coefficients. Third, the root cause of the high false alarm rate is not a modeling failure but an adversarial one: Infilteration attacks are engineered to be indistinguishable from normal traffic at the feature level, and no fusion strategy can contribute useful signal where none exists.

4.2 Comparison to Literature

Published results on CIC-IDS2018 frequently report detection accuracies of 95-99%. These figures are generally achieved on imbalanced datasets where the majority class dominates the test set, inflating accuracy because correct classification of the abundant benign records contributes disproportionately to the overall metric. Our evaluation uses a balanced dataset with 8,000 records per class in the test set, which is a substantially harder and more realistic evaluation. Under this balanced setting, 86.89% accuracy and 0.866 macro F1 represent meaningful multiclass discrimination across seven attack types. Our approach also contrasts with Dai et al. (2024) in its

fusion philosophy: rather than feeding reconstruction error as a raw feature into the classifier, we maintain strict agent independence and combine outputs probabilistically, which is more interpretable and more principled for handling disagreements between agents.

4.3 Limitations

The system has several honest limitations. A false alarm rate of 30.30% remains too high for real deployment, where security analysts are already overwhelmed by alert fatigue. DoS-SlowHTTPTest achieves only 52% recall in Agent 2, meaning approximately half of these attacks are missed. Agent 1's selective detection profile means the fusion improvement is concentrated in the attacks that are already handled well, providing no benefit where both agents simultaneously fail. The system was evaluated on a single dataset, and generalization to other network environments is unknown. Finally, the entire pipeline operates on flow-level features computed after a connection completes, which means it cannot provide real-time detection during an active attack flow.

4.4 Future Work

The most impactful improvement would be a stronger Agent 1 architecture. Replacing the feedforward autoencoder with an LSTM-based sequence model that operates on temporal patterns within network flows, rather than aggregate flow-level statistics, would likely produce substantially better separation for mimicry attacks, which operate at the behavioral level over time. Soltani et al. (2024) demonstrate that LSTM-based agents detect intrusions from the first 15 packets of a connection, suggesting substantial room for improvement in Agent 1's architecture. Feature engineering specifically targeting the Infilteration/Benign boundary, potentially incorporating behavioral heuristics such as session entropy or application-layer content signatures, represents the other major opportunity. At the architectural level, replacing the Logistic Regression

meta-model in stacking with a small neural network could capture nonlinear relationships between agent signals that the linear model misses. Finally, evaluation on additional public datasets such as NSL-KDD, UNSW-NB15, and CIC-IDS2017 would establish whether the system's performance generalizes across different network environments and collection methodologies.

Works Cited

Ahmad, I., Basheri, M., Iqbal, M. J., & Rahim, A. (2018). Performance comparison of support vector machine, random forest, and extreme learning machine for intrusion detection. IEEE Access, 6, 33789-33795. https://doi.org/10.1109/ACCESS.2018.2841987

Canadian Institute for Cybersecurity. (2018). CSE-CIC-IDS2018 on AWS. University of New Brunswick. https://www.unb.ca/cic/datasets/ids-2018.html

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785-794). https://doi.org/10.1145/2939672.2939785

Dai, Z., Por, L. Y., Chen, Y.-L., Yang, J., Ku, C. S., Alizadehsani, R., et al. (2024). An intrusion detection model to detect zero-day attacks in unseen data using machine learning. PLoS ONE, 19(9), e0308469. https://doi.org/10.1371/journal.pone.0308469

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. In Advances in Neural Information Processing Systems 32 (pp. 8024-8035). Curran Associates.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Soltani, M., Khajavi, K., Jafari Siavoshani, M., & Jahangir, A. H. (2024). A multi-agent adaptive deep learning framework for online intrusion detection. Cybersecurity, 7(1), Article 9. https://doi.org/10.1186/s42400-023-00199-0